Future Generation Computer Systems 93 (2019) 637–650

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

SAND: Social-aware, network-failure resilient, and decentralized microblogging system

Ding Ding [a,*], Mauro Conti [a], Renato Figueiredo [b]

[a] University of Padua, Italy
[b] University of Florida, US

HIGHLIGHTS

- In this paper, we propose SAND, a social-Aware, network failure-resilient, and decentralized microblogging system.
- SAND is the first to provide a high delivery rate (e.g., 100% in SAND-SN), with acceptable network overheads, in situations where a large portion (e.g., 90%) of network disconnected and users cannot access to the central servers of microblogging services.
- We propose and evaluate several routing algorithms to disseminate messages in SAND.
- We are the first to evaluate the performance of proposed decentralized microblogging system (i.e., SAND) on a ground-truth dataset with real social relationship, as well as real publisher–subscriber distribution.

ARTICLE INFO

Article history:
Received 26 February 2018
Received in revised form 12 September 2018
Accepted 7 November 2018
Available online 14 November 2018

MSC:
00-01
99-00

Keywords:
Decentralized microblogging
Distributed applications
Online social networking

ABSTRACT

To overcome the limitations (e.g., vulnerability to central server failure) of current existing microblogging systems (e.g., Twitter, and Weibo), researchers have proposed several decentralized microblogging solutions. However, these solutions also have several issues: (i) they were not designed for (and hence cannot handle) scenarios where massive correlated failures occur; or (ii) their delivery rates were not significantly high (i.e., lower than 85%); or (iii) their working mechanisms were not evaluated on partitioned networks based on a ground-truth dataset with a real publisher–subscriber distribution. In this paper, we propose SAND: a microblogging system builds upon an overlay where users have private IP connections to their social friends, enabling trusted social communication even on partitioned networks. We assess SAND through a simulation-based analysis considering a real-world dataset crawled from Twitter, as well as a synthetic dataset that provides high flexibility to tune network parameters. The results show that SAND is feasible and efficient, also in the case of network partitions. For example, in a partitioned network where 98% of network nodes fail, by using SAND-SN, peers are able to effectively follow each other's updates (i.e., 100% delivery rate). On average, the overhead is 1.9 copies, 22 network sends and 6.6 network receives per message.

1. Introduction

Microblogging services (such as Twitter, Weibo) have emerged as popular communication mechanisms in recent years. For example, Twitter – the most successful microblogging service – has attracted more than 335 million monthly active users as of Q2 2018.[1] Nowadays, microblogging services play important roles in many aspects of our social life, such as obtaining interesting news, and keeping in touch with friends. However, the current popular microblogging services are based on a client–server architecture, and are vulnerable to central server failure and/or censorship. In recent years, network failures happened where a localized portion of a network is partitioned from the rest of the Internet, due to natural hazards [1], mis-configurations [2], Distributed Denial of Service (DDoS) attacks on links [3], and national censorships [4]. For example, on the evening of January 27, 2011, Egypt –a country with a population of 80 million, including 23 million Internet users –was partitioned from the Internet. The connectivity of the network inside the country was recovered after five days. The people in Egypt were not able to use Twitter service because that Twitter servers are located outside of Egypt. Moreover, owing to the news media property [5] of microblogging services, under these extremely situations (e.g., censorships, and natural hazards) that result in network partitions, people are more willing to rely on this service, in order to spread information [6–8]. Therefore,

* Corresponding author.
E-mail address: austinxw@gmail.com (D. Ding).

[1] Twitter Monthly Active Users Crawl To 335M: https://www.statista.com/statistics/282087/number-of-monthly-active-twitter-users/.

https://doi.org/10.1016/j.future.2018.11.007
0167-739X/

researchers are called to design microblogging systems that are resilient to network failures. An intuitive solution to central server failures is to leverage decentralized Peer-to-Peer (P2P) networks. In the past few years, several decentralized microblogging solutions have been proposed. However, these solutions have the following issues: (i) they were not designed for (and hence cannot handle) scenarios where massive correlated failures occur; or (ii) their delivery rates were not significantly high; or (iii) they did not evaluated their working mechanisms on partitioned networks based on a real dataset, and with a real publisher–subscriber distribution.

We point out the following challenges to build a decentralized microblogging system:

- The system should support users with large numbers of subscribers (e.g., news media, celebrities), as well as users with few subscribers (e.g., a typical Internet user). For example, *katyperry* (a profile for the singer Katy Perry) in Twitter has over 76 million subscribers while the average number of subscribers in Twitter is only 208.[2] The design should be able to support users with different social patterns.
- The system should be able to work in situations where massive (e.g., 98%) nodes fail. Authors in [9] have already demonstrated that the routability of approaches based on Distributed Hash Table (DHT) (e.g., Chord [10]) is severely hampered in a massive nodes failure scenario. Therefore, the design should be able to support node bootstrapping and message routing without relying on DHT, in partitioned networks.
- The system should be able to achieve a desirable delivery rate. Publishers should find a way to deliver their messages to their subscribers.
- The system should support offline nodes that are inside a partitioned network. Since nodes inside the partitioned network may leave and re-join the network (e.g., devices restarted), or be censored by authority, the system needs to tolerate offline nodes even inside partitions.
- The system should allow the communication between users even under the constraints imposed by firewalls or Network Address Translation (NAT) [11].

In this paper, we propose SAND, a social-aware, network failure-resilience, and decentralized microblogging system. SAND is thought of as a possible alternative solution to centralized microblogging services (e.g., Twitter), and is able to provide the primary functions even in the situation of centralized microblogging services unavailable (i.e., network partitions where a large portion of network disconnected). SAND takes advantages of an available social P2P virtual private networking (SocialVPN [12]). By utilizing SocialVPN, SAND is built upon an overlay where users have private IP connections to their social friends. Consequently, SAND enables trusted social communication on partitioned networks (even through NATs and firewalls). It makes more difficult to censor, invade and disrupt, which consequently creates a network-failure resilient microblogging system. Since the social relationship is unstructured, SAND is resilient to network partitions, as well as offline nodes inside the partition area, without relying on DHT. Messages are stored in forwarders in order to ensure the delivery rate even when some of nodes in partitioned areas are offline. In order to study tradeoffs among different properties and delivery rates, we propose three variants of SAND: SAND based on Biased Random Walk algorithm (SAND-BR), SAND based on Popularity Flooding algorithm (SAND-PF), and SAND based on Super Node algorithm (SAND-SN). In particular, with SAND-BR and SAND-PF, by leveraging/improving Random Walk algorithm and considering the social network properties (e.g., the degree of a node, popularity of a message), we effectively improve the message delivery rate. In addition, with SAND-SN, we provide a novel idea by taking advantage of users with public IP addresses to help publishers disseminate messages. In order to avoid overheads on a single node with a substantial number of subscribers, the subscribers cannot establish direct links to their publishers, as long as they are friends to each other. Note that, the network costs of uploading, downloading and storing messages are important. These costs are modeled in our evaluation by considering the number of sent messages, number of received messages, and number of stored copies of messages, respectively.

Fig. 1 gives a bird's-eye view of the basic concept of SAND. In Fig. 1, the physical network level is at the bottom. It illustrates a physical distribution of endpoint devices and the links among them. Moreover, due to network failures, the devices inside the partitioned area cannot access the centralized server of a microblogging service. In the top part of Fig. 1, we represent the social relations among people who own the devices inside the partitioned area. People are able to communicate with their friends directly by leveraging SocialVPN. In SAND, instead of sending updates to central servers, a publisher sends messages to his or her subscribers in a P2P fashion. In Fig. 1, Alice is a **publisher**; Bob and Charlie are **subscribers** of Alice; other intermediary nodes may act as **forwarders** to help Alice to deliver messages. After a network partition, without central servers, Alice disseminates messages to her subscribers (i.e., Bob and Charlie) through one or more forwarders.

Contribution. In this paper, we propose SAND, a social-Aware, network failure-resilient, and decentralized microblogging system. To the best of our knowledge, SAND is the first to provide a high delivery rate (e.g., 100% in SAND-SN) in situations where a large portion (e.g., 90%) of network disconnected and users cannot access to the central servers of microblogging services. To deliver a message from a publisher to his subscribers, on average the network overhead is 1.9 copies, 22 network sends and 6.6 network receives per message. Moreover, we are the first to evaluate the performance of proposed decentralized microblogging system (i.e., SAND) on a ground-truth dataset with real social relationship, as well as real publisher–subscriber distribution.

Organization. The rest of the paper is organized as follows. Section 2 summarizes the related work. Section 3 describes the main design of SAND, and three variants of SAND: SAND-BR, SAND-PF and SAND-SN. In Section 4, we discuss the dataset used in the evaluation, and how we select a partitioned area. In Section 5, we first introduce the experimental setup, and then report the results of our evaluation. In Section 6, we first discuss how a user bootstraps into the overlay after partitions in SAND. Then we discuss the effect of offline nodes. Finally, we conclude the paper in Section 7.

2. Related work

In this section, we discuss the following topics in the state of the art: network partitioning, decentralized online social networks, structured decentralized microblogging systems, unstructured decentralized microblogging systems, and P2P virtual private networks.

Network partitioning. Recently, researchers have proposed several methodologies to analyze and address network partitions caused by events including country-wide censorships and natural disasters. In [4], Dainotti et al. utilized three types of data (Border Gateway Protocol updates, active trace route probing, and Internet

[2] Average number of subscribers (i.e., followers) per twitter user (Jeffbullas): http://www.jeffbullas.com/.

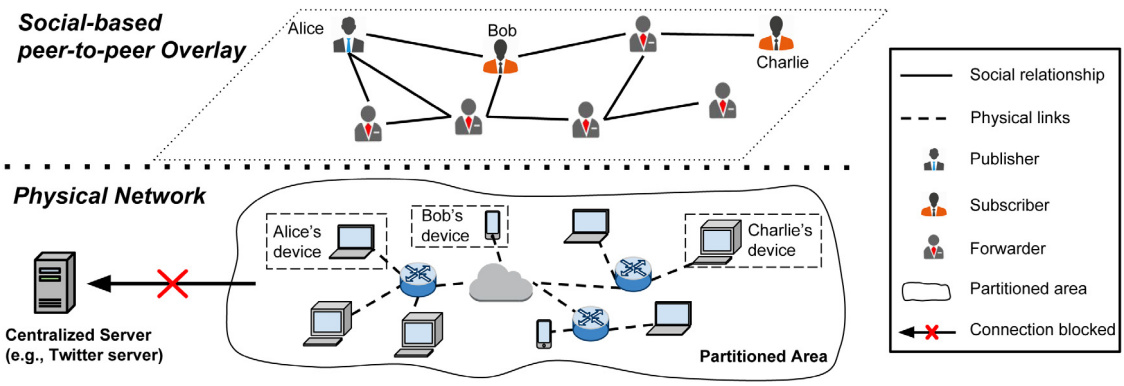


Fig. 1. Overview of SAND's message dissemination mechanism based on social-based P2P overlay.

background radiation traffic data) to analyze events that happened in two national Internet censorship episodes: in Libya and Egypt, in early 2011. The authors in [1] leveraged the malware pollution to analyze outage of Internet during two natural disasters — the recent earthquakes in New Zealand and Japan. These related works analyze the changes of network flow during the Internet outage. However, they did not propose a method to recover the connectivity at the application layer (e.g., for a microblogging application). In contrast, SAND focuses on the messages delivery at the application layer, inside the partitioned network. Software Defined Networking (SDN) approaches have also been investigated in the context of fault tolerance. In [13] and [14], the authors introduced SDN techniques to address node failure scenarios. They leveraged SDN architectures to recover network connectivity during node failure. However, SDN manages networks in a centralized way by means of an SDN controller. If the SDN controller is located in the partitioned area, it cannot control the rest of the network. Furthermore, SDN techniques require that a controller programs networking devices. While this assumption can be satisfied within an enterprise (e.g. a data center), it is not directly applicable when the devices are owned by different entities and distributed across the Internet. In order to provide people (inside partition area) access to blocked servers, authors in [15] proposed Pisces, a decentralized protocol for anonymous communications that leverages users' social links to build circuits for onion routing [16]. In addition, a user discovers peers by using random walks [17] in the social network graph. However, unlike SAND, Pisces does not consider message dissemination in a situation where large numbers of nodes fail (e.g., a partition).

Decentralized online social networking. Authors in [18] introduced the concept of Virtual Private Social Networks (VPSNs), and proposed FaceVPSN, a decentralized system which is able to enforce privacy protection in Facebook. FaceVPSN mitigates the problem of lack of privacy in Facebook and hides information from users outside the VPSN. In [19,20], authors proposed Distributed Online Social Networking (DOSN) Systems designed to provide OSN capabilities (e.g., the ability to post news, follow friends) in a decentralized way. PeerSoN [19] is a fully decentralized online social network. Peers use a DHT as a lookup service to locate each other's endpoints, then exchange encrypted information directly with each other. Safebook [20], based on DHT, focuses on anonymity by adding an additional layer of indirection to message requests by routing them through multiple layers of friends and friends of friends. However, the links in these networks are friend-to-friend links. They do not consider forwarding messages from a publisher to subscribers who are multiple-social-hops away from the publisher. Differently from the work in the state of the art, SAND focuses on disseminating messages from publishers to subscribers who might be not direct friends of each others.

Structured decentralized microblogging. In [21–23], authors proposed structured distributed microblogging systems. In particular, in Megaphone [21], subscribers join a multicast tree by using Pastry, a structured overlay based on DHT. In Cuckoo [22], peers discover each other's endpoints and send follow requests directly to each other through DHT. Publishers are then able to send updates directly to a subset of subscribers, depending on bandwidth availability, while the remaining subscribers use a gossip protocol to propagate updates among themselves. To handle churn, the Cuckoo approach relies on a centralized backend that stores all updates and follows requests from all users in case a publisher is not online. Twister [23] is comprised of three independent overlay networks: a Bitcoin protocol based overlay provides distributed user registration and authentication. DHT-based overlay provides key/value storage for user resources and tracker location for the third network. Bittorrent-based overlay is a collection of possibly disjoint "swarms" of subscribers, which can be used for efficient near-instant notification delivery to many users. However, Cuckoo [22] does not mention replication mechanism to ensure content availability considering offline users. Moreover, these previous works [19–23] are based on DHT, which has already been demonstrated in [9] that routability of approaches based on Distributed Hash Table (DHT) (e.g., Chord [10]) is severely hampered in a massive nodes failure scenario. In SAND, we leverage unstructured social relationship to disseminate messages from publishers to subscribers.

Unstructured decentralized microblogging. FETHR [24] aims at a fully decentralized HTTP-based microblogging service. Peers subscribe to each other by exchanging canonical URLs and use the HTTP GET and POST methods to pull or push updates to each other. FETHR uses gossip-based dissemination for users with a high number of subscribers. However, depending on URLs to push and pull messages is problematic when the DNS severs are not available. In addition, FETHR do not consider the networking constrains imposed by NATs or firewalls. Moreover, a partition event may cripple the relationship among subscribers, so that the message dissemination is hampered. In SAND-SN, instead of communicating by URLs, publishers and their subscribers communicate with intermediary forwarders by public IP address directly. MoP-2-MoP [25] is a decentralized privacy-preserving microblogging infrastructure based on a distributed peer-to-peer network of mobile users. User exchange messages based on local point-to-point communication links over a delay-tolerant opportunistic network. However, in order to disseminate messages to distant subscribers, publishers in this approach, need a central server that could be easily censored by authorities. Firechat[3] is a message dissemination application which has been used in network partition event.[4] It

[3] Firechat: http://opengarden.com/firechat/.

[4] Hong kong's 'off-grid' protesters: http://www.bbc.com/news/blogs-trending-29411159/.

delivers messages only according to the proximity among users. In SAND, publishers are able to send their messages precisely to their subscribers. Litter [26] is a P2P-based microblogging system that leverages the private IP connectivity of social-based VPN. It utilizes both IP multi-casting and Random Walks to ensure that publishers are able to delivery messages with varying degree of scope (i.e., friends, friends of friends, and/or the public). Considering the limited delivery rate of Random Walk, we propose SAND that provides improved message dissemination mechanisms. In addition, Litter evaluates its performance on a whole graph instead of a partitioned area. Therefore, the evaluation cannot reflect the performance during partition events. SAND is an alternative solution for current microblogging systems (e.g., Twitter, Weibo) and provides the primary functions in a situation where central servers fail (a portion of network partitioned from the Internet). In SAND, we perform an extensive simulation-based analysis to assess the delivery rate, as well as network overheads (i.e., transferred messages, stored copies of messages), in partitioned areas. Moreover, the evaluation in Litter is based on a synthetic dataset with friend-to-friend relationship, while does not consider a ground-truth publisher–subscriber distribution. In SAND, our simulation is based on a dataset of real data crawled from Twitter, that includes the ground-truth publisher–subscriber distribution. We summarize the related decentralized system in comparison with SAND in Table 1.

Peer-to-peer virtual private network (P2PVPNs). Recently, P2P virtual private networks (P2PVPNs) such as Hamachi,[5] SocialVPN [12] have become popular decentralized alternatives to centralized VPNs. However, in Hamachi, centralized Simple Traversal of UDP through NATs (STUN)-like servers are used to enable NAT traversal and establish direct P2P connections among users. In SocialVPN, NAT traversal is not centralized because it uses existing nodes in the overlay to perform UDP hole punching for direct P2P connectivity. In addition, SocialVPN leverages existing social infrastructures to enable VPNs. SAND is built upon an SocialVPN overlay where users have private IP connections to their social friends, which consequently enables trusted social communication on a partitioned network (even through NATs and firewalls). As shown in Fig. 1, in SAND, friends are able to communicate directly.

3. Our solution for microblogging: SAND

In this section, we present SAND — a social-Aware, network failure-resilient, decentralized microblogging system. For ease of exposition, we first introduce an overview of SAND (Section 3.1). Then, we describe the message dissemination mechanism (i.e., Random Walk algorithm) of Litter (Section 3.2). In addition, in order to increase the message delivery rate, we introduce three versions of SAND: SAND based on *Biased Random Walk algorithm* (SAND-BR), SAND based on *Popularity Flooding algorithm* (SAND-PF) and SAND based on *Super Node algorithm* (SAND-SN) (Sections 3.3 and 3.4). Finally, we discuss implementation issues (Section 3.5).

3.1. Overview

SAND is an alternative solution to centralized microblogging services (e.g., Twitter), in the situation where users cannot access to the central servers of the microblogging services. In particular, before a partition, nodes are able to exchange self-signed public key certificates, as well as detailed contact information to their direct friends, through central servers (e.g., Twitter's server). Once central servers fail, social friends can connect to each other to form a social-aware overlay [9] by leveraging the information gotten from central servers before the partition. SAND takes advantages of SocialVPN [12], and Litter [26] as its building blocks.

SAND is built upon a SocialVPN overlay where users have private IP connections to their social friends, and consequently enables trusted social communication on a network (even a partitioned network) through the constraints imposed by NATs and firewalls. By leveraging SocialVPN, social friends are able to communicate with each other. Note that, social information are obtained from the current online social networks (e.g., Facebook) before partition events. However, in a microblogging service, a message publisher not only needs to disseminate messages to his/her direct friends who follow him/her, but also needs to disseminate messages to his subscribers who may be multiple social hops away. Then, the challenge of designing SAND is how a publisher sends his messages to his social distant subscribers with acceptable network overheads. SAND is inspired from a previous solution, Litter (which uses a Random Walk algorithm) to design the message dissemination mechanisms. In our paper, in order to study and evaluate tradeoffs among different properties and delivery rates, we propose three variants of SAND: SAND-BR, SAND-PF, and SAND-SN. In the following, we first describe the concept of Random Walk algorithm used in Litter, then introduce the improved message dissemination algorithms used in SAND, respectively.

3.2. Basic dissemination mechanism in Litter

In Litter, a publisher disseminates messages using Random Walk algorithm (i.e., a publisher randomly pushes his messages to one or more forwarders in the network), while the subscribers pull messages by simply flooding to request their direct social friends.

In Random Walk, in order to disseminate messages to subscribers, a publisher pushes messages to the network hop by hop. In particular, a node (a publisher/forwarder) randomly selects one of his neighbors to forward messages. Note that, in Random Walk, a publisher's messages may be stored at nodes multiple social hops away from the publisher. The delivering scope of a message is controlled by a TTL (Time To Live): if a publisher wants to disseminate messages to one of the direct friends, then the TTL is set to 1. In order to enhance the delivery rate, TTL could be set to a large value (e.g., 50, 100) depending on the user's preference. However, selecting a higher TTL increases the chances that distant subscribers will be able to receive the updates of the publisher, while resulting in large network overheads (e.g., transferred messages, stored messages). The TTL serves as the replication factor because a messages is stored at each node the message reaches. Note that, if a node does not have a neighbor to forward a message, it will drop the message even the TTL is not 0.

In Litter, nodes in overlay are fully equal, and the overlay is resilient to nodes failures. However, the delivery rate in Litter under a partition event cab be improved. In order to enhance the delivery rate, in this paper, we propose following versions of SAND: *SAND-BR*, *SAND-PF*, and *SAND-SN*.

3.3. Random walk-based methods

In this section, we describe the message dissemination mechanisms based on random walk algorithm: SAND-BR (Section 3.3.1, and SAND-PF (Section 3.3.2).

3.3.1. SAND-BR (based on biased random walk)

A node with large degree has higher probability to be reached by other nodes [27] in the social graph. Therefore, we first propose a Cluster Walk algorithm: a node always forwards a message to a neighbor with largest degree. However, although Cluster walk achieves a desirable delivery rate, network overheads are higher

[5] Hamachi: https://secure.logmein.com/products/hamachi/.

Table 1
Comparison of SAND with the related distributed systems (*results are gotten based on the evaluation setup shown in Section 5)..

System	Type	Social-aware	Without relying on DHT	Network-failure resilient	Delivery rate in 2% partition
PeerSoN	Online social network	✗	✗	✗	0%
Safebook	Online social network	✓	✗	✗	0%
Megaphone	Microblogging	✗	✗	✗	0%
Cuckoo	Microblogging	✓	✗	✗	0%
FETHR	Microblogging	✗	✓	✗	*n/a*
Litter (*Random Walk*)	Microblogging	✓	✓	✓	84.9%*
FireChat	Message dissemination	✗	✓	✓	*n/a*
SAND-BR (Bias Random Walk)	**Microblogging**	✓	✓	✓	**87.8%***
SAND-PF (Popularity Flooding)	**Microblogging**	✓	✓	✓	**86.0%***
SAND-SN (Super Node)	**Microblogging**	✓	✓	✓	**100%***

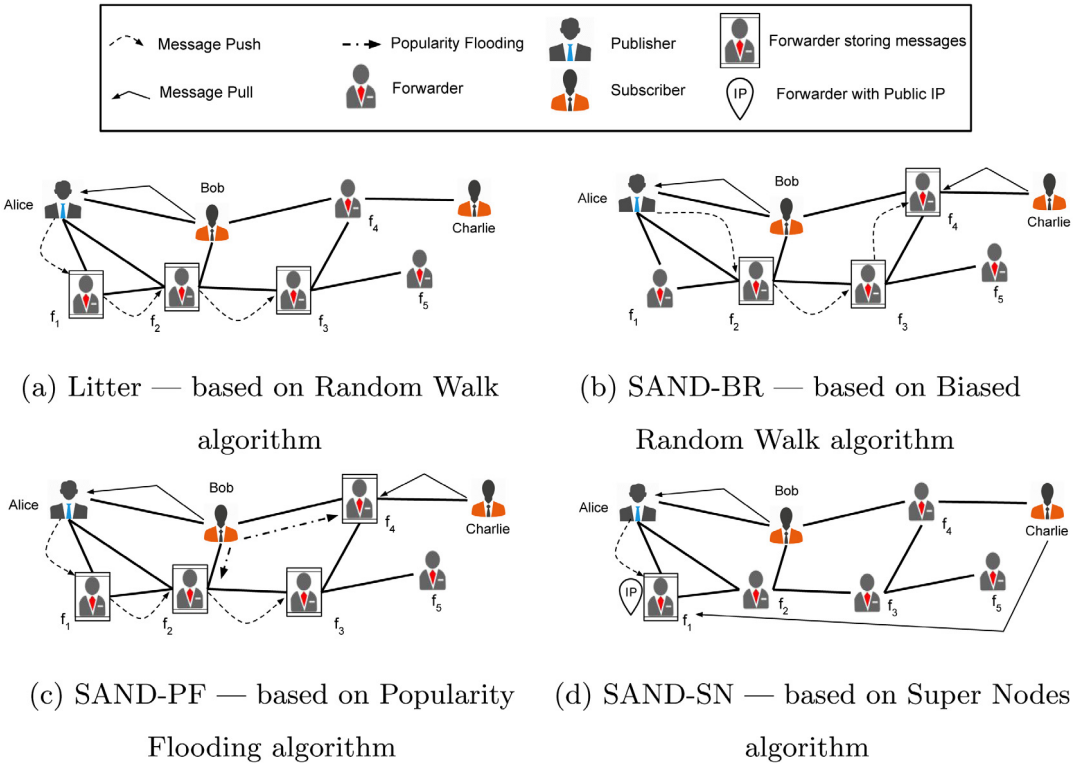


Fig. 2. Message dissemination mechanisms. Alice is a publisher; Bob and Charlie are subscribers of Alice.

for the set of nodes with relatively large degree. We then propose a tradeoff method (i.e., SAND-BR) based on Biased Random Walk algorithm, in which a node has higher probability to forward a message to a neighbor with large degree. Fig. 3 shows how Alice selects a neighbor to forward messages in Random Walk (Litter), Cluster Walk and Biased Random Walk (SAND-BR), respectively (topology shown in Fig. 2(b)). In Random Walk, all neighbors have equal chance to receive messages from Alice. In Cluster Walk, due to f_2 with largest degree, Alice only forwards messages to f_2. In Biased Random Walk, f_2 has highest probability to receive messages from Alice. Neighbors f_1 and *Bob*, nevertheless, still could receive messages from Alice.

Fig. 2(b) shows an example of how Alice disseminates messages to her subscribers in SAND-BR (TTL is set to 3). Different from Random Walk in Litter (shown in Fig. 2(a)), Alice forwards her message to f_2 in SAND-BR. Therefore, the message forwarding and storing path is $\{Alice, f_2, f_3, f_4\}$. Consequently, both Bob and Charlie can retrieve the message from Alice. Biased Random Walk algorithm is a tradeoff approach between Random Walk and Cluster Walk. It provides a high delivery rate, while keeping a relative uniformity of network overheads among nodes in a partitioned network. Note that, in SAND-BR, the forwarding and storing paths of a publisher's

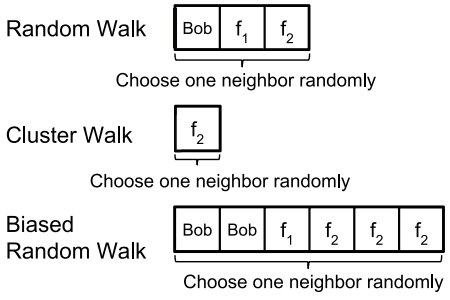


Fig. 3. Forwarding mechanism in Random Walk, Cluster Walk and Biased Random Walk algorithms, respectively.

messages may not be same. In Biased Random Walk, the *biased level* indicates how biased when a node sends a message to the neighbor with the largest degree (e.g., the probability that the message is sent to f_2 in the above example). It is possible to enlarge the *biased level* to increase this probability, and hence, to improve the delivery rate. This is because of the following two reasons: (1) sending

messages to nodes with larger degree indicates that the messages can be reached by more potential subscribers; (2) it decreases the probability of sending a message to a node without other neighbors in a partition. Although increasing *biased level* does not increase the overall networking overheads (i.e., stored messages, sent/received messages), the network overheads are not uniformly distributed in the network (i.e., network overheads are higher for the set of nodes with relatively large degree). The Cluster Walk is a special case of Biased Random Walk with largest biased level.

The procedure of message dissemination in SAND-BR is detailed in Algorithm 1. If the TTL of a message is 0, the node will store this message and stop forwarding (line 2 in Algorithm 1). Otherwise, the node chooses a forwarder from his forwarder list that contains the neighbors that have not stored the message yet (lines 5 and 6 in Algorithm 1). Once the node selects a forwarder from the list (by biased random selecting), the node sends the message to the forwarder. If the forwarder is able to receive this message, it will return a confirmation message to the node (lines 8–10 in Algorithm 1). Note that, a forwarder may not be available for two reasons: (i) the forwarder is located in outside of network partition area; (ii) the forwarder is offline. A *playload message* is a message successfully sent to forwarders; a *tentative message* is sent to forwarders unsuccessfully, because they are offline (or link failure); a *confirmation message* is sent by forwarders as an acknowledgment. If all nodes are available, successfully sending one payload message will consume one tentative message and one confirmation message. In contrast, if some nodes are unreachable, successfully sending one payload message will consume several tentative messages (based on the portion of offline nodes), and one confirmation message.

Algorithm 1 Publishers or forwarders disseminate messages in SAND-BR.

Input: *passedList* is a list containing all the forwarders that have the copies of the messages; *ttl* is to control the scope of the message; *message* is the message need to be forwarded.

```
1:  procedure PUSHMESSAGE(passedList, ttl, message)
2:      if ttl = 0 then
3:          return
4:      end if
5:      forwardersList ← publisher.neighbors()
6:      forwardersList.remove(passedList)
7:      while len(forwardersList)> 0 do
8:          forwarder ← biasedchoosing(forwardersList)
9:          forwardersList.remove(forwarder)
10:         flag ← sendMessage(forwarder, message)
11:         if flag = True then
12:             ttl ← ttl − 1
13:             passedList.append(forwarder)
14:             break
15:         end if
16:     end while
17: end procedure
```

3.3.2. SAND-PF (based on popularity flooding)

In this section, we introduce SAND-PF based on Popularity Flooding algorithm. Same with Litter and SAND-BR, a subscriber floods requests to his neighbors to retrieve messages. In order to enhance the delivery rate, once a subscriber receives a message, based on the popularity level [28] of the message (assess the characteristics of the message, as well as the publisher of the message), the subscriber will decide whether to flood the message to his neighbors.

In order to identify the popularity of a message, the authors in [29,30] discuss the following properties: *message content relevance features* (e.g., semantic analysis [31], time distance to the last message), *microblogging service specific features* (e.g., whether including URL, number of repost times), and *publisher authority features* (e.g., the number of subscriber of the message publisher, whether the message publisher is verified). In view of the decentralized characteristic of SAND, SAND-PF, we consider the following factors to decide whether a subscriber to flood a message:

- *Message content*: semantic analysis.
- *Publisher*: the number of subscribers; verified or not.
- *Surrounding situation*: the proportion of subscriber's neighbors who have the copies of the message.

Algorithm 2 shows the procedure of how to decide if a message is popular. A subscriber will flood a message, if the message satisfies the following requests: (i) the message is popular (by semantic analyzing, line 2 in Algorithm 2); (ii) enough subscribers follows this message (lines 3 and 4 in Algorithm 2); and (iii) the proportion of subscriber's neighbors who receive the message is low. By tuning *threshold1*, *threshold2*, and *threshold3*, SAND-PF is able to enhance delivery rate with increasing overall networking overheads.

Algorithm 2 Subscriber disseminates messages in SAND-PF.

Input: *receivedNeighbors* represents the neighbors who have the message's copies; *message* is the message to be flooded.
Note: *threshold*1 controls whether to flood messages according to contents of the messages; *threshold*2 controls whether to flood messages according to publishers' information; *threshold*3 controls whether to flood messages according to surrounding situation.

```
1:  procedure FLOODMESSAGE(receivedNeighbors, message)
2:      flag1 ← semanticAnalyzing(message) > threshold1
3:      subscribers ← message.owner().subscribers()
4:      flag2 ← len(subscribers) > threshold2
5:      portion ← len(receivedNeighbors)/len(neighbors)
6:      flag3 ← portion < threshold3
7:      if flag1&flag2&flag3 = True then
8:          sendMessage(message)
9:      end if
10: end procedure
```

With a view to the limitation of the simulation (the messages in the simulation do not include semantic content, as well as timestamps), we only consider the following metrics in our simulation: the number of subscribers (who published the message), and the proportion of neighbors (who have the copies of message). In particular, once a subscriber s receives a message posted by a publisher p, if p's number of subscribers is larger than a threshold, and the proportion of s neighbors who have the copies of the message is less than a threshold, s will flood the message to all his neighbors (the values of the thresholds are discussed in Section 5.1).

Fig. 2(c) shows an example of how Alice disseminates messages to her subscribers in SAND-PF (TTL is set to 3). The message forwarding and storing path is same as the one in Random Walk algorithm, $\{Alice, f_1, f_2, f_3\}$. Once Bob's overlay node receives the message from Alice, it decides that this message is meaningful. Moreover, it realizes that Alice has many subscribers, and the proportion of his neighbors who have the copies of this message is low. Under this circumstances, SAND-PF floods this message to all of Bob's neighbors. As a result, a copy of the message is also stored at f_4, where Charlie, another one of Alice's subscriber, is able to retrieve the message.

To avoid spam messages, a node may set a filter to block a set of popularity-based flooded message. Moreover, only subscribers assess messages to decide whether to flood, according to the characteristics of the messages. The reason for this behavior is: in SAND, forwarders only forward and store messages, in order to reduce the resources usage (i.e., computing).

3.4. Super node-based method: SAND-SN

In addition to the methods based on walk-based algorithms (i.e., Litter, SAND-BR, and SAND-PF), we make use of nodes with public IP addresses, and propose SAND-SN based on the concept of Super Node, which achieves better message delivery rate, while reducing the network overheads. Considering the fact that nodes with public IP addresses are minority in the current Internet [32], and such public IP nodes can provide more functionalities (i.e., other private IP nodes are able to communicate directly to them), we use the term "super nodes". Fig. 2(d) shows an example of how Alice disseminates messages to her subscribers in SAND-SN. Before the partition, each publisher (e.g., Alice) maintains a list, named Super Nodes List (SNL), that contains a set of nodes with public IP address within a certain social hop count (e.g., within 2 social hops). Each of Alice's subscribers has a local copy of Alice's SNL. After network partitions, according to her SNL, Alice sends messages to the public-IP nodes where Alice's subscribers could retrieve the messages. Alice forwards her messages to f_1. Given that f_1 has public IP address, other subscribers (i.e., Charlie) of Alice are able to connect to f_1 directly to retrieve the message from Alice.

One situation needs to be cautiously considered in SAND-SN is offline status of super nodes. In SAND-BR and SAND-PF, we do not consider failed nodes or offline nodes, since all nodes work together with equal roles. However, super nodes may be unavailable even inside partition networks for three reasons: (i) super nodes themselves may be offline; (ii) the overheads on some of the super nodes can become significant, so that super nodes cannot serve requests from publishers and subscribers; (iii) nodes with public IP address are more vulnerable to censorship. Therefore, as a consequence, the following situation might happen: a publisher could only reach a portion of super nodes in his SNL.

As shown in Fig. 4, Alice is a publisher and Bob is one of her subscribers. Note that, in Fig. 4, we only consider Bob as Alice's only existing subscriber; s_1, s_2, s_3 and s_4 are four super nodes in Alice's SNL. Although the ID of the last message Alice posts is 42, due to the offline status of s_1, s_2, s_3 and s_4, several messages are absent from these super nodes. For example, the messages with the IDs ranging from 37 to 41 stored in s_1 are missed. The reason of this behavior is that, s_1 is offline when Alice posts messages with the IDs ranging from 37 to 41. To mitigate a "message missing" problem caused by nodes' offline status, in SAND-SN, we propose a push mechanism and a pull mechanism for publisher and subscriber, respectively.

Publisher. When pushing messages, in order to guarantee enough super nodes to receive messages, a publisher considers a threshold, indicating the proportion of super nodes that are online and could receive messages. The publisher will stop sending messages to the super node as long as the proportion of super nodes who receive messages is beyond the threshold. Algorithm 3 shows how a publisher pushes a message to his super nodes. First, the publisher randomly selects a super node from his SNL (lines 3 and 4 in Algorithm 3). Then the publisher sends his message to the super node. If the super node is available, he will send a confirmation message to the publisher (line 5 in Algorithm 3). Note that, this confirmation message is similar to the one we discussed in Section 3.3.1. If the portion of super nodes who have the message's copy is larger than a threshold, the publisher will stop pushing messages (line 9 in Algorithm 3).

Subscriber. When a subscriber receives a message whose ID is not consistent with previous one (which means the subscriber has missed some messages), the subscriber will traverse all the super nodes (in the SNL of a node who generates that message) to request the missed messages. Algorithm 4 shows how a subscriber retrieve messages if the subscriber misses one or more messages. First, the subscriber sends a request message to a super node (the super node is randomly selected from the SNL of the publisher who sends that message) to request recent messages generated by the publisher (lines 4–7 in Algorithm 4). Once the subscriber receives the messages from the super node, he updates his messages list to check whether the missed message is retrieved (lines 8 and 9 in Algorithm 4). Note that, the subscriber may find more missed messages in this procedure. If the subscriber still misses one more messages, he will send a request message to another super node. Otherwise, if he receives all the messages, he will stop sending messages to super nodes. In Fig. 4, after retrieving messages from s_1, Bob realizes that some of Alice's messages (i.e., ID from 37 to 42) are absent. Then Bob retrieves these missed messages from s_2, s_3 and s_4, respectively. In order to balance the loads on super nodes, SAND-SN leverages the following methods: (1) a publisher randomly choose super nodes to forward messages; (2) a super node is able to set a threshold to accept messages from up to "n" publishers.

Algorithm 3 A Publisher disseminates messages to forwarders in SAND-SN.

Input: *snList* is the Super Node List of the publisher; *message* is the message need to send to super nodes.
Note: If the portion of super nodes who receive messages larger than the *threshold*, the publisher stop disseminating messages.

```
1:  procedure pushMessage(snList, message)
2:      count ← 0
3:      shuffle(snList)
4:      for each sn ∈ snList do
5:          flag ← sendMessage(sn, message)
6:          if flag = True then
7:              count ← count + 1
8:          end if
9:          if count/len(snList) > threshold then
10:             break
11:         end if
12:     end for
13: end procedure
```

Algorithm 4 A Subscriber retrieves messages from forwarders in SAND-SN.

Note: *message* is the received message.

```
1:  procedure pullMessage( )
2:      missedMessageList ← MissedMessage(message)
3:      if len(missedMessageList) > 0 then
4:          publisher ← message.owner()
5:          snList ← publisher.snl()
6:          shuffle(snList)
7:          for each sn ∈ snList do
8:              messagesList ← retrieveMessages(sn, publisher)
9:              missedMessageList.update(messagesList)
10:             if len(missedMessageList) = 0 then
11:                 break
12:             end if
13:         end for
14:     end if
15: end procedure
```

SAND-SN obtains 100% delivery rate (based on the evaluation setup shown in Section 5.2) by leveraging nodes with public IP addresses. In SAND-SN, public-IP nodes become relay servers that store and distribute messages. The total network overheads are lower than the one in SAND-BR and SAND-PF; while the delivery rate of SAND-BR is able to reach 100%.

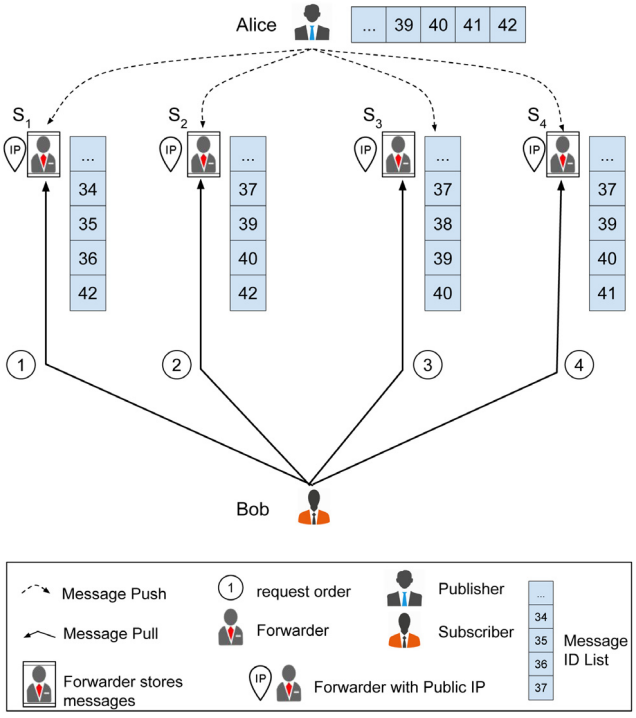


Fig. 4. SAND-SN. Forwarders s_1, s_2, s_3, and s_4 are four super nodes that store and forward Alice's messages. Each super node maintains a list that contains the messages from Alice, ordered by message ID. Message ID is an integer that increments by one for each message published by a publisher. Assume Bob successfully pulls the messages with the IDs that are less than 34.

In Section 1, we pointed out five challenges to build a decentralized microblogging system. In the following, we discuss how SAND variants are able to address these challenges: (1) In stead of establishing links between publishers and subscribers, with SAND, publishers "push" messages into network (i.e., one or more forwarders store such messages). By leveraging this idea, with SAND, a publisher with a large number of subscribers is able to avoid to establish a large number of connections to all their subscribers. (2) In order to work in a situation where massive nodes fail, SAND is not designed based on DHT. We leverage unstructured overlay to disseminate messages in a decentralized way. (3) Section 5 shows that, with SAND, subscribers are able to receive messages with a desirable rate. (4) With SAND, publishers disseminate messages into a network by randomly (or biased randomly) selecting forwarders from their neighbors (or neighbors within n hops in SAND-SN). This mechanism ensures publishers only choose online nodes at each message sending process. If a offline node goes online, the node will become the candidate who might receive messages from publishers. (5) By leveraging socialVPN, users with SAND are able to communicate even behind NAT or firewalls.

3.5. Implementation

By leveraging SocialVPN[6] and based on the prototype of Litter,[7] we implement the following functionalities in SAND: object serialization, data encryption, data transmission, data storage, and web interface. Note that, we use SQLite for the data storage. Our implementation is written in Python. Fig. 5 shows the user interface of SAND.

To communicate, we created a service that listens on a user-specified UDP port for incoming messages or requests. To retrieve messages, the request payload is encapsulated in a UDP datagram that is sent to the multi-cast IP address (social friends) through SocialVPN, so that the listening service is able to receive the request message. Note that, because microblogging messages are less than 140 characters, such messages are able to capsulate one UDP message without IP fragmentation. SocialVPN maintains the friendship list/SNL for users.

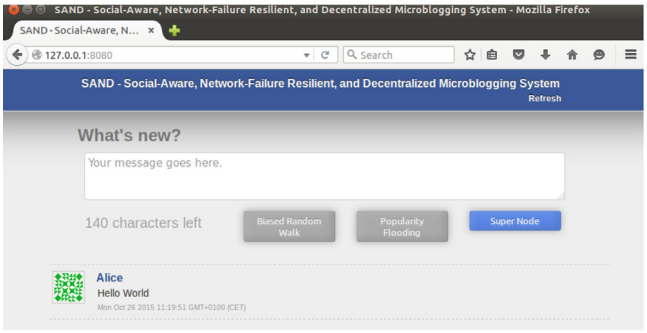


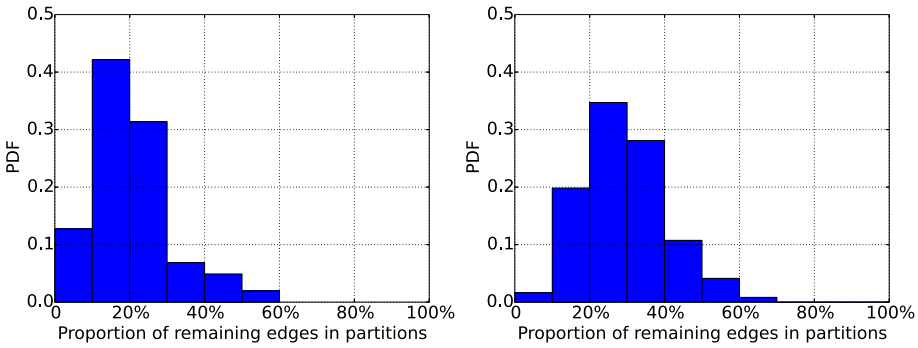
Fig. 5. Interface of SAND.

4. Dataset

We are the first to simulate and analyze the performance of the proposed decentralized microblogging system on a dataset with real publisher–subscriber distribution, in a partitioned network. In addition, we also run our evaluation based on a synthetic dataset. This section describes the datasets used in our evaluation. We obtained a Twitter dataset from [33]. The raw data consist of 456,626 users and 14,855,842 directed edges. If there are bi-directional edges between two users, we can assume they are friends to each other; otherwise, they are just a publisher and a subscriber.

Considering that this Twitter dataset only includes 456,626 nodes, it may not show a comprehensive behavior of a system that

6 The IPOP Project (Github): https://github.com/ipop-project/.
7 Litter (Github): https://github.com/pstjuste/litter/.



(a) Partition with 2% nodes (b) Partition with 4% nodes

Fig. 6. Proportion of remaining edges in partitions. The *y*-axis represents probability distribution function (PDF). The results are obtained from 100 simulation runs.

has similar scale as Twitter (billions of users). Therefore, we also use a synthetic dataset in our evaluation since it is able to provide more flexibility (i.e., different number of total nodes in a network). We generate a social network (including nodes and their friendships) based on a modified Nearest Neighbor Model [34]. Note that we create the synthetic datasets using metrics that proportionally approximate those found in the Twitter dataset.

We then assign location information for the nodes based on the following concepts: "the shorter is the distance among two nodes, the higher is the probability for them to be friends". Based on the location, we cut a certain area to simulate network partition events.

Authors in [35] show that, if a country partitioned from the whole social network graph, on average 84.2% percent of edges will remain within the country. However, due to the limitations of the synthetic algorithm (i.e., *Loc-Generation*), after partitioning, the range of proportions of remaining edges in partitions are from around 10% to around 50%. Fig. 6 shows the distribution of the proportion of remaining edges in partitions. In particular, partitions include 2% nodes in Fig. 6(a), while partitions include 4% nodes in Fig. 6(b). As shown in Fig. 6, the average proportion of remaining edges in 4% partition is larger than the one in 2% partitions. This is because the larger the number of nodes in the partition, the higher the possibility for path to exist between nodes and their neighbors [9].

To be more realistic, we select partitions with more than 40% remaining links in the following evaluation. Note that, this is still a conservative number, based on the discussion on the previous paragraph.

5. Evaluation

In this section, we perform a simulation-based evaluation to assess the delivery rate and the network overheads (i.e., transferred messages, stored messages) of SAND. In particular, we describe the experimental setup in Section 5.1. Then we report the results in Section 5.2. The evaluation is analyzed with the NetworkX graphing library,[8] which is a Python package for examining complex networks.

5.1. Simulation setup

In our evaluation, we first analyze the *delivery rate* of SAND in different size of networks, and in different size of partitions. In addition, as we discussed in Section 1, we take censorship in Egypt as a study case. In our simulation, within a Egypt-scale partition (i.e., 2%), we analyze the detailed probability distribution of *delivery rate*, *stored messages* and transferred messages (i.e., *sent messages* and *received messages*) in a simulation run. In the following, we define the metrics reported in our evaluation.

- *Delivery Rate*: For a publisher i, it posts x_i messages ($0 < x_i \leqslant 5$). For a message, we assume q ($0 < q \leqslant m$) subscribers receive it (m subscribers follow the publisher). Note that messages may be disseminated by different paths, so that the subscribers who receive the messages may be different in different message dissemination turns. Therefore, the publisher's delivery rate r_i is:

  $$r_i = \frac{\sum_{j=1}^{x} q_j}{m_i x_i}$$

  In addition, assume a network have n publishers. The delivery rate of network R is:

  $$R = \frac{\sum_{i=1}^{n} r_i}{n}$$

- *Stored Messages*: When a forwarder receives a message from a publisher or another forwarder, the forwarder first stores the message, then forwards the message based on the remaining TTL. In our simulation, we analyze the probability distribution of messages stored on each node in partitioned networks. Note that the number of stored messages reflects the number of used TTL in SAND-BR. While in SAND-SN, this number indicates the number of super node in publishers' SNL after partitions.
- *Sent Messages*: In our evaluation, we analyze the number of sent messages and received messages respectively. In particular, we assess the probability distribution of sent messages on each node in a partitioned network in a simulation run.
- *Received Messages*: We assess the probability distribution of received messages on each node in a partitioned network in a simulation run. Note that the number of received messages of a node may be different from the number of sent messages of the node. The reason for this behavior is: the node may send a message to a forwarder/super node who is located outside of a partition. In this case, the node cannot receive the reply messages from the outside forwarder/super node.

Authors in [5] show that nodes with large number of subscribers, or following large numbers of publishers, have higher probability to post more message. In the evaluation, each node, as a publisher, sends 1–5 messages according to the number of his subscriber and number of publishers he follows. For Litter, SAND-BR and SAND-PF, TTL is set to 100, in the Twitter-dataset evaluation. In the synthetic-dataset evaluation, TTL changes according to the total number of nodes in the network. In SAND-PF, the threshold of

[8] Networkx: http://networkx.github.io/.

Table 2
Input parameters.

Parameters	Value
Range of generated messages per publisher	1–5
TTL (Litter, SAND-BR and SAND-PF)	100
Threshold of neighbors with message copies (SAND-PF)	30%
Threshold of number of subscribers (SAND-PF)	20
Proportion of Public-IP nodes (SAND-SN)	20%
Total items in SNL (SAND-SN)	20
Maximum hops of super nodes in SNL (SAND-SN)	3

neighbors with the copies of a messages is 30%, while the threshold of the publisher's (who posts the messages) number of subscribers is 20. According to [32], and considering increasing prevalence of IPv6, the portion of public-IP nodes accounts for 20% (out of overall nodes), in SAND-SN. In addition, the number of super nodes in SNL is 20, and the maximum hops when a node selects super nodes in SNL is 3. Table 2 shows the input value we considered in our evaluation.

5.2. Simulation results

In this section, we first show the delivery rate in the different size of networks (proportions of partition are same). Then, we show the delivery rate in different size of partitions (the initial number of nodes in networks are same). In addition, because network costs are important factors that need to be considered, we then report the distribution of delivery rate, as well as network overheads (i.e., stored messages, sent messages and received messages) in the partition with 2% nodes. Note that, for the following metrics, we report the average of the results obtained for the 50 simulation runs.

5.2.1. Delivery rate in different size of networks

In this section, we show the delivery rates of Litter, SAND-BR, SAND-PF and SAND-SN in networks with different initial nodes (the proportions of partition are same). Since we analyze the delivery rates in the situation where the number of nodes in the initial network changes, this evaluation runs on top of the synthetic dataset.

Fig. 7(a) shows changes in average delivery rate for different size of networks. Note that, the proportions of partitions are 2% in this evaluation. When the number of nodes in the initial network increases, the delivery rates of Litter, SAND-BR, and SAND-PF fluctuate around 84%, 86% and 87%, respectively. This is because these walk-based mechanisms are fully distributed, and they do not rely on any central server. Note that, as discussed in Sections 3.3.1 and 3.3.2, SAND-BR and SAND-PF can achieve higher delivery rate by tuning the biased level and the thresholds of Algorithm 2. The delivery rate of SAND-SN is 100%. This is because that, publishers are always able to find public-IP forwarders who could distribute messages to subscribers.

Since the hardware limitations of our clusters used for this evaluation, we cannot run this evaluation on a network with larger number of nodes. In this evaluation, censored area includes 2% nodes of total network nodes. Therefore, there are 200,000 nodes in the censored area when the initial network includes ten million nodes. While in a realistic scenario, Egypt has 519,000 Twitter users.[9] Although the number of censored nodes in the simulation is smaller than the number of Egyptian Twitter users, it is on the same order of magnitudes as for the Egyptian Twitter users.

Note that, since the following evaluations do not need to change the number of nodes in the initial network, all these evaluations runs on top of the Twitter dataset.

[9] Report:http://www.arabsocialmediareport.com/Twitter/LineChart.aspx.

5.2.2. Delivery rate in different size of partitions

Fig. 7(b) shows changes of average delivery rate, considering partitions with different numbers of nodes. The partition size is from 2% to 18%. We choose these numbers because: (1) our study case – Egypt – includes around 2% world population; (2) China – another censorship-victim country has most population in the world – includes around 18% world population. Note that, according to [9], all DHT-based microblogging solutions (e.g., Megaphone, Cuckoo) are not able to disseminate messages (i.e., delivery rate is 0) in the partitions whose size is less than 10%. With SAND, the delivery rate is able to reach over 86% in such size of partitions (the delivery rate is able to achieve 100% with SAND-SN). In Litter, SAND-BR, and SAND-PF, the delivery rates slightly increase when the size partitions increases. The reason for this behavior is: as shown in Section 4, a partition with larger size, has higher probability to maintain large proportion of remaining edges. As a result, publishers have higher probability to find delivery paths to forward messages to subscribers in large size partitions.

Moreover, the average delivery rates in SAND-BR and SAND-PF are higher than the one in Litter. The reasons for this behavior are: (i) in SAND-BR, a node with higher degree has higher probability to be a forwarder to store and transfer messages, and a large-degree forwarder connects more nodes, therefore subscribers have higher probability to reach a large-degree forwarder in order to retrieve the messages. (ii) In SAND-PF, a subscriber will diffuse a message if the message is 'popular'. As a result, the subscribers of the message have higher probability to get this message. Note that, as we discussed in Section 4, the proportion of remaining edges in partition network in our simulation is lower than the one in reality. Therefore, the results shown in Fig. 7(b) are still conservative. We believe in a real usage scenario, the delivery rate could be even better.

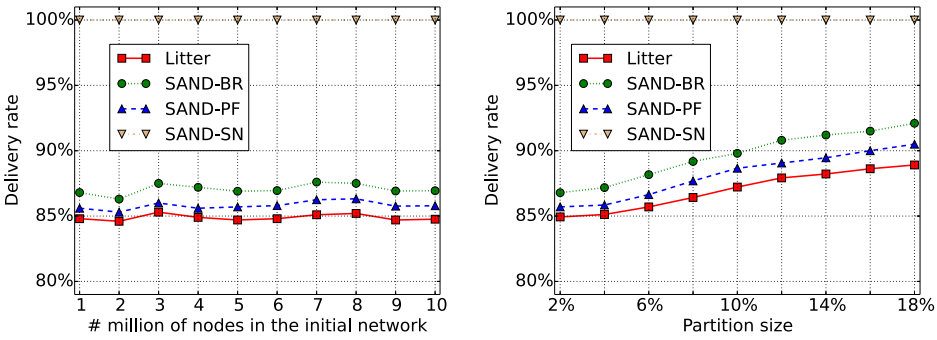
In SAND-SN, all the nodes in partition networks achieve 100% delivery rate. The reason for this behavior is that, all publishers are able to find at least a forwarder with Public IP address to forward messages to their subscribers. Note that, the delivery rate in practice may not be this high for two reasons: (i) proportion of public IP address is different from country to country (from area to area). A publisher is not able to find a public IP node within a short social hop in a partition area; (ii) A public IP forwarder is not available even inside a partition area due to censorship, or excessive forwarding requests.

5.2.3. Detailed evaluation of a partition with 2% nodes

In this subsection, we report the distribution of delivery rate, the number of stored messages, the number of sent messages, and the number of received messages, respectively, in a partition with 2% nodes.

Distribution of delivery rate. Fig. 8(a) shows the delivery rate distribution in Litter, SAND-BR, SAND-PF, and SAND-SN, respectively. The Delivery rate of Random Walk, Biased Random Walk, Popularity Flooding and Super Node are: 84.9%, 87.8%, 86.0% and 100%, respectively. In particular, for around 60% of the nodes, the delivery rates achieves 100% in Litter, SAND-BR, and SAND-PF, while all the nodes in SAND-SN have 100% delivery rate.

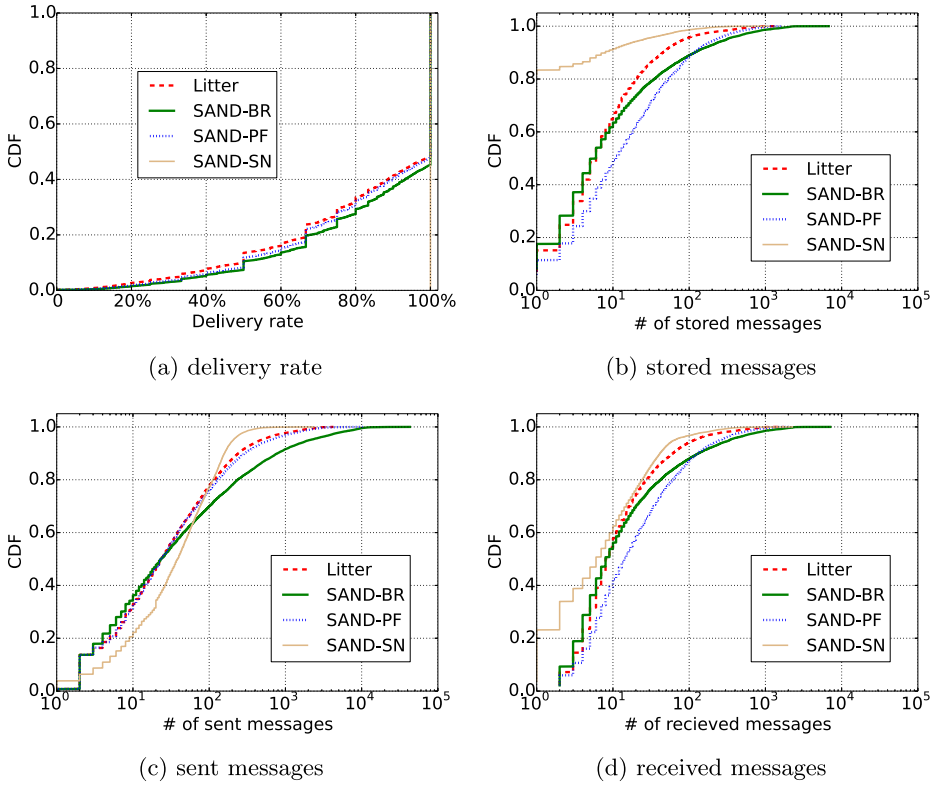
There are noticeable discontinuities in the lines that represent Litter, SAND-BR and SAND-PF. They occur at $x = 1/2$, $x = 2/3$, $x = 3/4$, and $x = 4/5$. The reason for this behavior is that the publishers who have 2, 3, 4 or 5 subscribers, although unable to deliver their messages to all of their subscribers, are able to deliver the message to most of their subscribers (only one subscriber cannot be delivered).



(a) **Delivery rate in different size of networks.** (number of nodes in the initial network ranging from 1 million to 10 million; synthetic dataset)

(b) **Delivery rate in different size of partitions** (partitions with 2%, 4%, 6%, 8%, 10%, 12%, 14%, 16% and 18% nodes, respectively; Twitter dataset)

Fig. 7. Analysis of delivery rate in different size of networks, and in different size of partitions.



(a) delivery rate

(b) stored messages

(c) sent messages

(d) received messages

Fig. 8. Detailed evaluation of a partition with 2% nodes. The Partition includes 10,000 nodes. The *y*-axis represents cumulative distribution function (CDF). In Figs. 8(b)–8(d), the *x*-axis is in log scale.

Distribution of number of stored messages. Fig. 8(b) shows the distributions of number of stored messages on nodes in a partitioned network in a simulation run, in Litter, SAND-BR, SAND-PF, and SAND-SN, respectively. Specifically, the number of stored messages in SAND-BR and SAND-PF is larger than the one in Litter. The reason for this behavior is: in SAND-BR, a node always has higher probability to send a message to a forwarder with large degree, which indicates nodes have higher probability to find a forwarder before TTL exhausted. As a result, more nodes hold the copies of messages in SAND-BR. Note that, the nodes with a small degree in SAND-BR have smaller probability to be a forwarder. As shown in Fig. 8(b), in SAND-BR, there are more number of nodes with less than seven messages copies, and more number of nodes with large number of messages copies (more than 2000). Note that large number of messages on a node may result in node resource exhaustion and failure, and thus needs to be mitigated.

In SAND-PF, after a message dissemination process from publishers to forwarders, once a subscriber receives a message, the subscriber may also flood the message to his neighbors. Therefore, as shown in Fig. 8(b), the number of messages in Popularity Flooding is always higher than the one in Random Walk algorithm.

In SAND-SN, only forwarders (super nodes) in publishers' SNL store messages. Although a publisher sends his messages to several forwarders (super nodes), this sending process is only in one hop. In other words, a forwarder does not send messages to another forwarder. As a result, the number of stored message is low. In

addition, only the nodes with public IP address can be the forwarders. In our simulation, the public IP proportion is set to 20%. Therefore, as shown in Fig. 8(b), more than 80% nodes do not store any message (few public IP node cannot be reached by publishers). Note that, the length of SNL, the Proportion of public IP address, and the size of partitions are other factors that affect the number of stored messages in SAND-SN. The length of SNL in our simulation is set to 20. The number of stored messages will increase if the length of SNL becomes larger.

Distribution of number of sent and received messages. In the transferred messages analysis, we assess sent messages and received messages separately, in a simulation run. Fig. 8(c) shows the distribution of the number of sent messages in Litter, SAND-BR, SAND-PF, and SAND-SN, respectively. In SAND-SN, there is a noticeable change at $x = 20$. The reason for this behavior is that, the length of SNL in our simulation is 20. For the nodes who do not follow publishers, they only send 20 messages to their super nodes to publish messages.

Fig. 8(d) shows the distribution of the number of received messages in Litter, SAND-BR, SAND-PF, and SAND-SN, respectively. Nodes send/receive less messages in SAND-SN compared to Litter, SAND-BR, and SAND-PF. The reason of this behavior is that, for the node with private IP address in SAND-SN, they cannot serve as forwarders. As a result, the private-IP nodes send/receive messages when they are publishers (i.e., send post to forwarders, receive confirmation messages from forwarders) or subscribers (i.e., send request to forwarders, retrieve messages from forwarders). However, in Litter, SAND-BR, and SAND-PF, every node can be a forwarder if their neighbors send them forwarding requests.

Nodes in SAND-BR and SAND-PF send/receive more messages compared to Litter. The motivations for this behavior are: (i) in SAND-BR, a message has higher probability to use more TTL when it is forwarded in a partitioned network (as shown in Fig. 8(b), TTL serves as the replication factor because a messages is stored at each node the message reaches). (ii) In SAND-PF, after a message dissemination from publishers to subscribers, subscribers also may flood the message to their neighbors according to the popular level of the message.

Fig. 8(b) shows the distribution of number of stored messages, which indicates the number of payload messages sent to forwarders successfully. In contrast, Fig. 8(c) and Fig. 8(d) show all the messages (including payload messages, tentative messages and confirmation messages) sent and received in the partitioned network. Comparing Fig. 8(b) with Fig. 8(c) and Fig. 8(d), we can have a clue that number of tentative messages is larger the number of payload messages. This is because in the 2% size of partitioned network, over 98% nodes fail. Hence, a publisher/forwarder may try several times to successfully send a payload message.

5.3. Experiment results

Having evaluated the delivery rate and the costs of sending/receiving messages, we now carry out experiments to analyze a prototype that implements the core overlay virtual networking capability of SAND. In particular, we carry on the experiment of our prototype to show the ICMP latency between two nodes. One node is in Alibaba's ECS cloud (located in eastern China),[10] and the another node is in Cloudlab (located in U.S.).[11] The experiment assumes these two nodes are connected, and evaluates the ICMP latency between them. Note that, we report the results obtained across 50 experimental runs.

We report the minimum value, maximum value, median value and average value of ICMP latency in Table 3.

Table 3
Results of ICMP latency.

	Min	Max	Median	Average
latency	189 ms	190 ms	190 ms	189.79 ms

As we can see in Table 3, the average latency between two SAND node is 189.79 ms. Note that, in our experiment, one node is located in China and another node is located in U.S.. The reason why we run our experiment on the nodes in such locations is the following: the latency between such nodes is a relatively large one, because of the distance between these locations. Therefore, this is a conservative number. In the real scenario, it is more likely a latency between two nodes are smaller than this number.

6. Discussion

In this section, we first discuss a possible solution for how a node bootstraps into the overlay after network partitions in SAND. Next, we discuss the issue of offline nodes.

Bootstrapping. Bootstrapping needs to be considered when network partition occurs in SAND. Although this is not within the scope of this paper, we have an approach that is able to assist nodes in partition network to bootstrap into the overlay even under the constraints imposed by firewalls and NATs. In order to assist nodes with private IP address to bootstrap into the overlay, nodes with public IP address may serve as STUN or Traversal Using Relays around NAT (TURN) servers. STUN servers allow NATed nodes to connect directly in a P2P fashion, while TURN servers allow a relayed link through an intermediary. In order to connect to STUN or TURN servers (i.e., public-IP nodes) and to discover other peers after a partition, each node maintains a boot list, created before the partition, which contains information of several public nodes. Note that, public nodes in one's boot-list are one's nearest (shortest social-hops) nodes with public IP address. If network partition happens, according to the boot-list, each node connects to one of the "servers" in order to bootstrap. As shown in Section 5, all private IP nodes are able to find "servers" to bootstrap into the overlay, after partitions.

Offline nodes. SAND disseminates and store messages based on a social-aware overlay [9] where people are able to communicate with their social friends directly. Therefore, in the case of offline forwarders (inside partitioned area), nodes could still communicate with other online forwarders. In other words, if the first choice is offline, a node can select an alternative choice. Furthermore, given that forwarders are able to store messages, even a subscriber offline during a message dissemination, the subscriber still could retrieve the message once it becomes online (as long as the subscriber has at least one direct friend who is a forwarder). Note that, 'offline' is not a permanent status. When a node becomes online, it will re-establish all the connections with other online social friends. Authors in [36] leverage the concept of 'personal overlay' to connect all devices that belongs to one person. A personal overlay represents a node (a person) in our system. Although one individual device (e.g., smartphone) may be offline at some point, there can be other online devices (e.g., laptop, iPad) belonging to the same user who is online and connects to our system. Therefore, our system is connected by personal overlays to personal overlays. Moreover, people tend to own more and more personal online devices [32], and the online time of devices tends to increase, considering that data plans become increasingly accessible to end users. As a result, SAND is resilient to offline nodes even the nodes are inside the partitioned area.

[10] Alibaba ECS: https://www.aliyun.com/product/ecs.
[11] Cloudlab: https://www.cloudlab.us.

Trusted communications among social friends. SAND is based on a social overlay built by social relationships, where only social friends are able to communicate with each other. This provides a mechanism that can prevent unauthorized users to join the system and send spam messages. However, in practice, an unauthorized user with a fake profile may be able to establish a friendship with other users. Detecting unauthorized users (e.g., fake profiles) and spam messages are out of the scope of this paper.

7. Conclusion

This paper proposes a microblogging system that allows messages to be delivered in the situation of network partitions and node failures. The system – SAND – builds upon a social peer-to-peer overlay where users have private IP connections to their social friends, enabling trusted social communication even on partitioned networks. In particular, we propose three different variants of SAND: SAND-BR, SAND-PF, and SAND-SN. We assess SAND through a simulation-based analysis using a Twitter dataset as well as a synthetic dataset. Our simulation results show that SAND is feasible and efficient, also in case of network partitions. For example, in a partitioned network with 2% remaining nodes, the delivery rate are able to achieve 100% with SAND-SN. On average, the overhead is 1.9 copies, 22 network sends and 6.6 network receives per message. The experimental results show that the delay between nodes in SAND is less than 200 ms.

Acknowledgments

This work is partially supported by the EU TagItSmart! Project (agreement H2020-ICT30-2015-688061), the EU-India REACH Project (agreement ICI+/2014/342-896), and the grant n. 2017-166478 (3696) from Cisco University Research Program Fund and Silicon Valley Community Foundation.

This material is based upon work supported in part by the National Science Foundation under Grants No. 1527415 and 1339737. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

References

[1] A. Dainotti, R. Amman, E. Aben, K.C. Claffy, Extracting benefit from harm: using malware pollution to analyze the impact of political and geophysical events on the Internet, ACM SIGCOMM Comput. Commun. Rev. 42 (1) (2012) 31–39.
[2] Z. Nabi, The anatomy of Web censorship in Pakistan, 2013, ArXiv preprint arXiv:1307.1144.
[3] M.S. Kang, S.B. Lee, V.D. Gligor, The crossfire attack, in: 2013 IEEE Symposium on Security and Privacy (S&P), IEEE, 2013, pp. 127–141.
[4] A. Dainotti, C. Squarcella, E. Aben, K.C. Claffy, M. Chiesa, M. Russo, A. Pescapé, Analysis of country-wide internet outages caused by censorship, in: Proceedings of the 2011 ACM SIGCOMM Conference on Internet Measurement Conference, IMC, ACM, 2011, pp. 1–18.
[5] H. Kwak, C. Lee, H. Park, S. Moon, What is Twitter, a social network or a news media? in: Proceedings of the 19th International Conference on World Wide Web, WWW, ACM, 2010, pp. 591–600.
[6] G. Lotan, E. Graeff, M. Ananny, D. Gaffney, I. Pearce, et al., The Arab Spring — the revolutions were tweeted: Information flows during the 2011 Tunisian and Egyptian revolutions, Intl. J. Commun. 5 (2011) 31.
[7] A. Choudhary, W. Hendrix, K. Lee, D. Palsetia, W.-K. Liao, Social media evolution of the Egyptian revolution, Commun. ACM 55 (5) (2012) 74–80.
[8] K. Starbird, L. Palen, (How) will the revolution be retweeted?: information diffusion and the 2011 Egyptian uprising, in: Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work, CSCW, ACM, 2012, pp. 7–16.
[9] D. Ding, M. Conti, R. Figueiredo, Impact of country-scale Internet disconnection on structured and social P2P overlays, in: 2015 IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks, WoWMoM, IEEE, 2015, pp. 1–9.
[10] I. Stoica, R. Morris, D. Liben, D.R. Karger, M.F. Kaashoek, F. Dabek, H. Balakrishnan, Chord: a scalable peer-to-peer lookup protocol for internet applications, Trans. Netw. (2003) 17–32.
[11] L. D'Acunto, J. Pouwelse, H. Sips, A measurement of NAT and firewall characteristics in peer-to-peer systems, in: Proc. 15-th ASCI Conference, vol. 5031, Advanced School for Computing and Imaging (ASCI), 2009, pp. 1–5.
[12] P. Juste, D. Wolinsky, P. Oscar, M. Covington, R. Figueiredo, SocialVPN: Enabling wide-area collaboration with integrated social and overlay networks, Comput. Netw. (2010) 1926–1938.
[13] A. Aydeger, N. Saputro, K. Akkaya, S. Uluagac, Assessing the overhead of authentication during SDN-enabled restoration of smart grid inter-substation communications, in: Consumer Communications & Networking Conference (CCNC), 2018 15th IEEE Annual, IEEE, 2018, pp. 1–6.
[14] A. Capone, C. Cascone, A.Q. Nguyen, B. Sanso, Detour planning for fast and reliable failure recovery in SDN with OpenState, in: Design of Reliable Communication Networks (DRCN), 2015 11th International Conference on the, IEEE, 2015, pp. 25–32.
[15] P. Mittal, M. Wright, N. Borisov, Pisces: anonymous communication using social networks, in: NDSS, 2013.
[16] R. Dingledine, N. Mathewson, P. Syverson, Tor: The second-generation onion router, Tech. Rep., DTIC Document, 2004.
[17] L. Katzir, S.J. Hardiman, Estimating clustering coefficients and size of social networks via random walk, ACM Trans. Web 9 (4) (2015) 19:1–19:20, http://dx.doi.org/10.1145/2790304.
[18] M. Conti, A. Hasani, B. Crispo, Virtual private social networks and a facebook implementation, ACM Trans. Web 7 (3) (2013) 14.
[19] S. Buchegger, D. Schiöberg, L.H. Vu, A. Datta, PeerSoN: P2P social networking: early experiences and insights, in: Proceedings of the Second ACM EuroSys Workshop on Social Network Systems, 2009, pp. 46–52.
[20] L.A. Cutillo, R. Molva, M. Önen, Safebook: a distributed privacy preserving online social network, in: 2011 IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks, WoWMoM, IEEE, 2011, pp. 1–3.
[21] T. Perfitt, B. Englert, Megaphone: fault tolerant, scalable, and trustworthy p2p microblogging, in: 2010 Fifth International Conference on Internet and Web Applications and Services, ICIW, IEEE, 2010, pp. 469–477.
[22] T. Xu, Y. Chen, J. Zhao, X. Fu, Cuckoo: towards decentralized, socio-aware online microblogging services and data measurements, in: Proceedings of the 2nd ACM International Workshop on Hot Topics in Planet-scale Measurement, HotPlanet, 2010, p. 4.
[23] M. Freitas, twister-a P2P microblogging platform, 2013, ArXiv preprint arXiv:1312.7152.
[24] D. Sandler, D.S. Wallach, Birds of a FETHR: open, decentralized micropublishing, in: The 8th International Workshop on Peer-to-Peer Systems, IPTPS, 2009, p. 1.
[25] M. Senftleben, M. Bucicoiu, E. Tews, F. Armknecht, S. Katzenbeisser, A.R. Sadeghi, Mop-2-mop–mobile private microblogging, in: Financial Cryptography and Data Security, Springer, 2014, pp. 384–396.
[26] P. St.Juste, H. Eom, K. Lee, R. Figueiredo, Enabling decentralised microblogging through P2PVPNs, in: The 10th Annual IEEE Consumer Communications and Networking Conference, CCNC, IEEE, 2013, pp. 323–328.
[27] A. Mislove, M. Marcon, K.P. Gummadi, P. Druschel, B. Bhattacharjee, Measurement and analysis of online social networks, in: Proceedings of the 7th ACM SIGCOMM conference on Internet measurement, IMC, ACM, 2007, pp. 29–42.
[28] P. Bao, H.-W. Shen, J. Huang, X.-Q. Cheng, Popularity prediction in microblogging network: a case study on sina weibo, in: Proceedings of the 22Nd International Conference on World Wide Web, WWW '13 Companion, ACM, 2013, pp. 177–178, http://dx.doi.org/10.1145/2487788.2487877.
[29] Y. Duan, L. Jiang, T. Qin, M. Zhou, An empirical study on learning to rank of tweets, in: Proceedings of the 23rd International Conference on Computational Linguistics, 2010, pp. 295–303.
[30] L. Hong, O. Dan, B.D. Davison, Predicting popular messages in twitter, in: Proceedings of the 20th International Conference Companion on World Wide Web, WWW, ACM, 2011, pp. 57–58.
[31] X. Hu, L. Tang, J. Tang, H. Liu, Exploiting social relations for sentiment analysis in microblogging, WSDM '13, ACM, 2013, pp. 537–546, http://dx.doi.org/10.1145/2433396.2433465.
[32] D. Evans, The internet of things: how the next evolution of the internet is changing everything, CISCO White Paper 1 (2011).
[33] M. De Domenico, A. Lima, P. Mougel, M. Musolesi, The anatomy of a scientific rumor, Nature Sci. Rep. 3 (2013).
[34] A. Sala, L. Cao, C. Wilson, R. Zablit, H. Zheng, B.Y. Zhao, Measurement-calibrated graph models for social network experiments, in: Proceedings of the 19th International Conference on World Wide Web, ACM, 2010, pp. 861–870.
[35] J. Ugander, B. Karrer, L. Backstrom, C. Marlow, The anatomy of the facebook social graph, 2011, ArXiv preprint arXiv:1111.4503.
[36] R.J. Figueiredo, S. Aditya, K. Jeong, K. Subratie, Seamless networking among edge devices and clouds with fog social virtual networks, in: Sensor to Cloud Architectures Workshop, SCAW, with HPCA, 2015.

Ding Ding received the Ph.D. degree in computer science from University of Padua, Italy, in 2017. In 2015, he was a Visiting Researcher (working with Prof. Renato Figueiredo) in the advanced computing and information systems laboratory (The ACIS Lab), University of Florida, USA. After his Ph.D., he was a research fellow in New York Institute of Technology (NYIT). His research interests include social P2P networking, cybersecurity.

Mauro Conti is Full Professor at the University of Padua, Italy, and Affiliate Professor at the University of Washington, Seattle, USA. He obtained his Ph.D. from Sapienza University of Rome, Italy, in 2009. After his Ph.D., he was a Post-Doc Researcher at Vrije Universiteit Amsterdam, The Netherlands. In 2011 he joined as Assistant Professor the University of Padua, where he became Associate Professor in 2015, and Full Professor in 2018. He has been Visiting Researcher at GMU (2008, 2016), UCLA (2010), UCI (2012, 2013, 2014, 2017), TU Darmstadt (2013), UF (2015), and FIU (2015, 2016). He has been awarded with a Marie Curie Fellowship (2012) by the European Commission, and with a Fellowship by the German DAAD (2013). His research is also funded by companies, including Cisco and Intel. His main research interest is in the area of security and privacy. In this area, he published more than 250 papers in topmost international peer-reviewed journals and conference. He is Area Editor-in-Chief for IEEE Communications Surveys & Tutorials, and Associate Editor for several journals, including IEEE Communications Surveys & Tutorials, IEEE Transactions on Information Forensics and Security, and IEEE Transactions on Network and Service Management. He was Program Chair for TRUST 2015, ICISS 2016, WiSec 2017, and General Chair for SecureComm 2012 and ACM SACMAT 2013. He is Senior Member of the IEEE.

Renato Figueiredo is a Professor at the Department of Electrical and Computer Engineering of the University of Florida. Dr. Figueiredo received the B.S. and M.S. degrees in Electrical Engineering from the Universidade de Campinas in 1994 and 1995, respectively, and the Ph.D. degree in Electrical and Computer Engineering from Purdue University in 2001. From 2001 until 2002 he was on the faculty of the School of Electrical and Computer Engineering of Northwestern University at Evanston, Illinois, and from 2012 to 2013 he was a visiting researcher at Vrije Universiteit, the Netherlands. His research interests are in the areas of virtualization, distributed systems, overlay and software-defined networks, cloud and edge computing, and their applications in support of computational science in domains including lake ecology, bio-diversity, and smart and connected communities. Dr. Figueiredo's research team leads the IPOP (IP-over-P2P) open-source overlay virtual network project.